

LIBRARY MANAGEMENT SYSTEM

MINOR PROJECT

Group Head : Durfishan

SUBMITTED BY:

- 1.DURFISHAN**
- 2.ABIRA**
- 3.AFREEN**
- 4.HURAT**
- 5.BINT-UL-HUDA.**

THREE YEAR DIPLOMA COMPUTER ENGINEERING



**GOVERNMENT POLYTECHNIC FOR WOMEN, SRINAGAR
DEPARTMENT OF COMPUTER ENGINEERING**

CO-ORDINATOR Sign

EXAMINAR Sign

INTRODUCTION OF PYTHON

Python is a high-level, versatile programming language known for its simplicity and readability. It facilitates efficient code development with a vast ecosystem of libraries and frameworks. Its clean syntax and dynamic typing make it popular for diverse applications, including web development, data analysis, artificial intelligence, and automation.

WHAT IS A LIBRARY MANAGEMENT SYSTEM

A Library Management System is software designed to manage library resources, including books, patrons, and transactions. It tracks inventory, handles book checkouts and returns, maintains user records, and aids librarians in cataloging, organizing, and efficiently managing library operations for optimal resource utilization and user services.



THE

LIBRARY



Libraries are vital as they offer access to knowledge, resources, and diverse information. They promote literacy, education, and lifelong learning, fostering intellectual growth, critical thinking, and cultural enrichment within communities. Libraries serve as hubs for research, innovation, and societal development, benefiting individuals and society at large.

INTRODUCTION OF LIBRARY MANAGEMENT SYSTEM

The Python-based Library Management System acts as a comprehensive tool merging the capabilities of MySQL for data storage and command-line interface for user interaction. It effectively administers book inventory, monitors book transactions, and provides various functionalities accessible through a command-line interface.

DESCRIPTION:

This software establishes a seamless connection to a MySQL database, enabling efficient interaction with a 'lmanage' database, housing essential book-related data. Each book entry encompasses crucial information like name, code, total count, and subject, all meticulously stored within the 'books' table.

FUNCTIONALITIES AND FLOW:

On the beginning of program , we have to import mysql connector and some important configuration to connect to the database create using xampp

```
import mysql.connector
mydb = mysql.connector.connect(
    host = "localhost",
    user = "root",
    password = "",
    database = "1manage",
)
```

We can add something different also , if we face an exception error .

```
import mysql.connector

config = {
    'host': 'localhost', # Hostname or IP address of the MySQL server
    'port': 3307,        # Port number
    'user': 'root',
    'password': '',
    'database': '1manage'
}

try:
    # Establish connection
    mydb = mysql.connector.connect(**config)
    print("Connected to MySQL database!")
except mysql.connector.Error as error:
    print("Error connecting to MySQL:", error)
```

1.Add Book:

Users input book details including name, code, count, and subject via the command line. The system then utilizes SQL queries to insert this information into the 'books' table within the MySQL database.

EXAMPLE

```
def addbook():
    bname = input("Enter book name: ")
    bcode = input("Enter book code:")
    total = input("Total books: ")
    sub = input("Enter subject:")
    data = (bname,bcode,total,sub)
```

```

sql = "insert into books values(%s,%s,%s,%s)"
c = mydb.cursor()
c.execute(sql,data)
mydb.commit()
print("-----")
print("Data entered successfully")
main()

```

```

def addbook():
    bname = input("Enter book name: ")
    bcode = input("Enter book code:")
    total = input("Total books: ")
    sub = input("Enter subject:")
    data = (bname,bcode,total,sub)
    sql = "insert into books values(%s,%s,%s,%s)"
    c = mydb.cursor()
    c.execute(sql,data)
    mydb.commit()
    print("-----")
    print("Data entered successfully")
    main()

```

2.Issue and Submit Books:

Two distinct functions, namely issueb() and submitb(), serve the purpose of handling book transactions. The former function captures book issuance while the latter manages book submissions. These functions prompt users to input necessary details such as name, roll number, book code, and date. Upon entry, they update the book count using the bookup() function.

EXAMPLE

```

def issueb():
    name = input("Enter name:")
    rno = input("Enter rno:")
    code = input("Enter book code:")
    date = input("Enter date:")
    sql = "insert into issue values(%s,%s,%s,%s)"
    data = (name, rno, code, date)
    c = mydb.cursor()
    c.execute(sql, data)
    mydb.commit()
    print("-----")
    print("book issued to :",name)
    bookup(code,-1)

```

```
def issueb():
    name = input("Enter name:")
    rno = input("Enter rno:")
    code = input("Enter book code:")
    date = input("Enter date:")
    sql = "insert into issue values(%s,%s,%s,%s)"
    data = (name, rno, code, date)
    c = mydb.cursor()
    c.execute(sql,data)
    mydb.commit()
    print("-----")
    print("book issued to :",name)
    backup(code,-1)
```

EXAMPLE

```
def submitb():
    name = input("Enter name:")
    rno = input("Enter rno:")
    code = input("Enter book code:")
    date = input("Enter date:")

    sql = "insert into submit values(%s,%s,%s,%s)"
    data = (name,rno,code,date)
    c = mydb.cursor()
    c.execute(sql,data)
    mydb.commit()
    print("-----")
    print("book submitted from :",name)
    backup(code,1)
```

```
def submitb():
    name = input("Enter name:")
    rno = input("Enter rno:")
    code = input("Enter book code:")
    date = input("Enter date:")
    sql = "insert into submit values(%s,%s,%s,%s)"
    data = (name,rno,code,date)
    c = mydb.cursor()
    c.execute(sql,data)
    mydb.commit()
    print("-----")
    print("book submitted from :",name)
    backup(code,1)
```

3.Update Book Count:

The bookup() function manages the adjustment of book counts in the database. It ensures accurate inventory records by retrieving the current count of a book, modifying it based on the transaction type (issue or submit), and updating the count within the 'books' table.

EXAMPLE

```
def bookup(code,u):
    sql = "select TOTAL from books where BCODE = %s"
    data = (code,)
    c = mydb.cursor()
    c.execute(sql,data)
    myresult = c.fetchone()
    t = myresult[0] + u
    sql = "update books set TOTAL = %s where BCODE = %s"
    d = (t,code)
    c.execute(sql,d)
    mydb.commit()
    main()
```

```
def bookup(code,u):
    sql = "select TOTAL from books where BCODE = %s"
    data = (code,)
    c = mydb.cursor()
    c.execute(sql,data)
    myresult = c.fetchone()
    t = myresult[0] + u
    sql = "update books set TOTAL = %s where BCODE = %s"
    d = (t,code)
    c.execute(sql,d)
    mydb.commit()
    main()
```

4.Delete Book:

Through the dbook() function, users can remove a specific book entry from the database by inputting its code. This function uses an SQL query to delete the specified book record.

EXAMPLE

```
def dbook():
    ac = input("Enter nook code:")
    sql = "delete from books where BCODE = %s"
```

```

data = (ac,)
c = mydb.cursor()
c.execute(sql,data)
mydb.commit()
main()

```

```

def dbook():
    ac = input("Enter nook code:")
    sql = "delete from books where BCODE = %s"
    data = (ac,)
    c = mydb.cursor()
    c.execute(sql,data)
    mydb.commit()
    main()

```

5.Display Books:

The dispbook() function retrieves all book details from the 'books' table within the database and showcases them in a structured format on the command prompt. This allows users to view the available inventory efficiently.

EXAMPLE

```

def dispbook():
    sql = "select * from books"
    c = mydb.cursor()
    c.execute(sql)
    myresult = c.fetchall()
    for i in myresult:
        print("book name:",i[0])
        print("book code:",i[1])
        print("Total:",i[2])
        print("-----")
    main()

```

```

def dispbook():
    sql = "select * from books"
    c = mydb.cursor()
    c.execute(sql)
    myresult = c.fetchall()
    for i in myresult:
        print("book name:",i[0])
        print("book code:",i[1])
        print("Total:",i[2])
        print("-----")
    main()

```

6.In Main() function:

It displays a menu of options for library management tasks, prompts the user to choose a task , based on the users input it causes the appropriate functions to perform the selected tasks, if the user enters an invalid choice, it prompts the user again.

```
def main ():
    print(""" .....LIBRARY MANAGEMENT.....
          1. Add book
          2. Issue book
          3. Submit book
          4. Delete book
          5. Display book
          """)
    choice = input("Enter task no:")
    print("-----")
    if(choice == '1'):
        addbook()
    elif(choice == '2'):
        issueb()
    elif(choice == '3'):
        submitb()
    elif(choice == '4'):
        dbook()
    elif(choice == '5'):
        dispbook()
    else:
        print("wrong choice")
        main()
```

7.Password() function:

This function generates a random password, let the user to enter a username , prints the generated password , asks the user to enter the generated password. If the entered password matches the generated password , it causes the main function. If the entered password doesn't match , it prompts the user again.

```

def password():
    import random
    ps = random.randint(000000,100000)

    user = input("Enter Username:")
    print("Your password is:",ps)

    verify = input("Enter password")

    if verify == str(ps):
        main()
    else:
        verify != str(ps)
        print("wrong password")
        password()

password()

```

USER INTERACTION

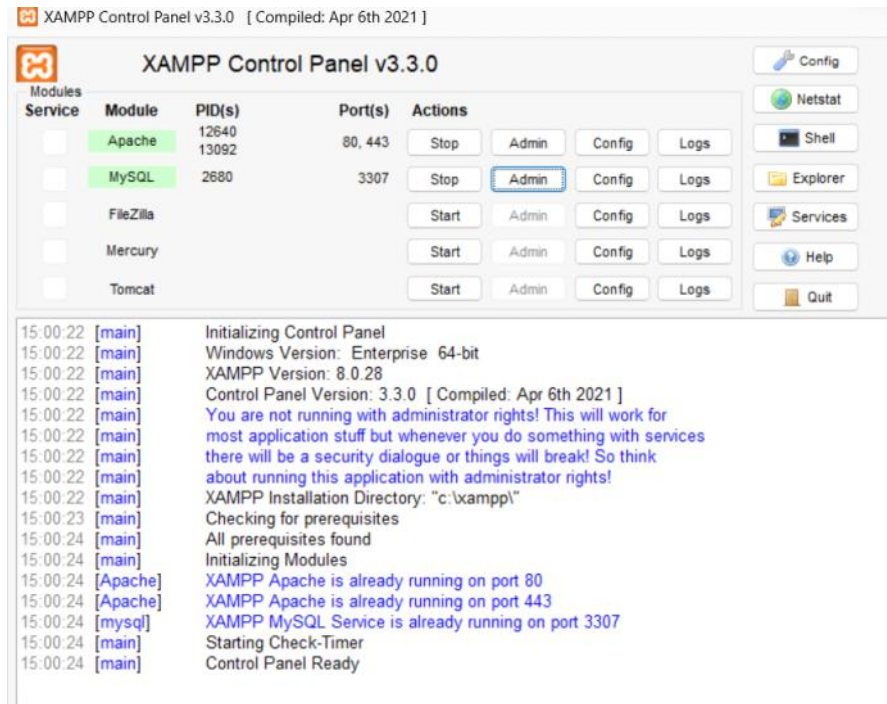
The system's user interaction occurs through a command-line interface, presenting users with a menu-driven system. This interface allows users to select their desired operations by entering numerical input. Based on user choices, the program dynamically executes corresponding functionalities. Additionally, the system incorporates a password verification system, limiting access to authorized users and ensuring security in managing library tasks.

software requirements

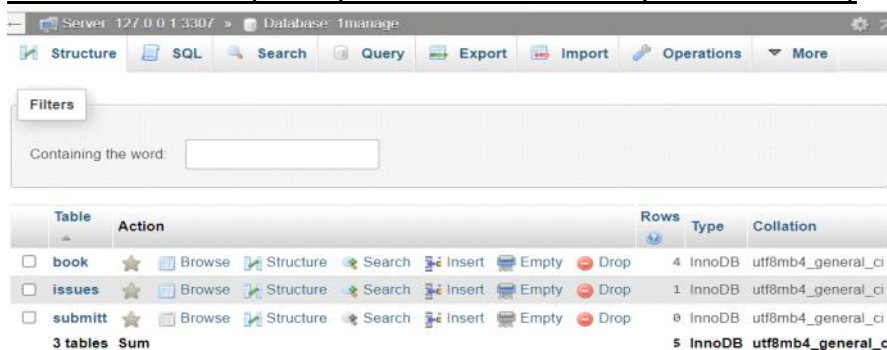
1. Python.
- 2.Xampp(mysql).
- 3.Vs Code.
- 4.Chrome or any another browser.

Creating Database

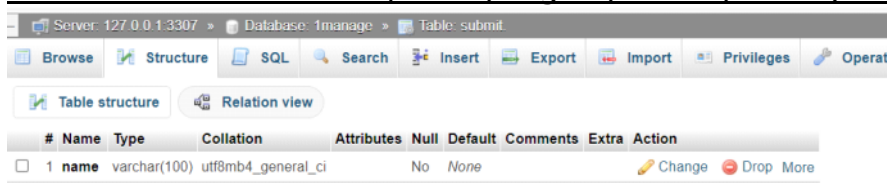
To create a database ,we have to start the mysql on xampp and click on the admin . After clicking , the PhpMyAdmin page will be displayed . Now here we can create a databases.



Here the Table (book, issues and submit) are created;



Inside the submit Table (name, regno ,bcode , sdate) rows are created;



Server: 127.0.0.1:3307 » Database: 1manage » Table: submit

Browse Structure SQL Search Insert Export Import Privileges Operations

Table structure Relation view

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	name	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
☐ 2	regno	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
☐ 3	bcode	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
☐ 4	sdate	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More

Inside the issue Table(name, regno, bcode, idate) rows are created;

Server: 127.0.0.1:3307 » Database: 1manage » Table: issues

Browse Structure SQL Search Insert Export Import Privileges Operations

Table structure Relation view

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	name	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
☐ 2	regno	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
☐ 3	bcode	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
☐ 4	idate	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More

Inside the book Table(bname, total, bcode, subject) rows are created;

Server: 127.0.0.1:3307 » Database: 1manage » Table: book

Browse Structure SQL Search Insert Export Import Privileges Operations

Table structure Relation view

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	bname	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
☐ 2	bcode	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
☐ 3	total	int(100)			No	None			Change Drop More
☐ 4	subject	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More

Here is the step wise description of the process in command prompt:

STEP 1:

```
C:\>cd\

C:\>cd xampp

C:\xampp>cd mysql

C:\xampp\mysql>cd bin

C:\xampp\mysql\bin>cd mysqld
The system cannot find the path specified.

C:\xampp\mysql\bin>cd -u root -p -h localhost
The system cannot find the path specified.
```

Here in the command prompt, the command cannot, due to the port error problem . So, we forcefully execute the command by entering the user , password and even the port number .

STEP 2:

```
C:\xampp\mysql\bin>mysql --user=root --password= --port=3307
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 488
Server version: 10.4.28-MariaDB mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

STEP 3:

```
mysql> show databases;
+-----+
| Database |
+-----+
| lmanage  |
| information_schema |
| mysql    |
| ofbsphp  |
| ofbsphp.new |
| performance_schema |
| phpmyadmin |
| test     |
+-----+
8 rows in set (0.00 sec)

mysql> use lmanage
Database changed
mysql> show tables;
+-----+
| Tables_in_lmanage |
+-----+
| books              |
| issue              |
| submit             |
+-----+
3 rows in set (0.00 sec)
```

STEP 4:

```
mysql> describe books;
ERROR 1932 (42S02): Table 'lmanage.books' doesn't exist in engine
mysql> describe book;
+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+
| bname | varchar(100) | NO   |     | NULL    |       |
| bcode | varchar(100) | NO   |     | NULL    |       |
| total | int(100)     | NO   |     | NULL    |       |
| subject | varchar(100) | NO   |     | NULL    |       |
+-----+
4 rows in set (0.04 sec)

mysql> describe issues;
+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+
| name  | varchar(100) | NO   |     | NULL    |       |
| regno | varchar(100) | NO   |     | NULL    |       |
| bcode | varchar(100) | NO   |     | NULL    |       |
| idate | varchar(100) | NO   |     | NULL    |       |
+-----+
4 rows in set (0.03 sec)
```

STEP 5:

```
mysql> describe submitt;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| name  | varchar(100)  | NO   |     | NULL    |       |
| regno | varchar(100)  | NO   |     | NULL    |       |
| bcode | varchar(100)  | NO   |     | NULL    |       |
| sdate | int(100)      | NO   |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)

mysql> select *from book;
Empty set (0.00 sec)
```

After executing all the above steps , we have to open the text editor console where we have already written the program for running the code . It shows an output as 'Enter username' , and shows password right bellow there, and allows user to enter this password again , here the authentication process been completed. Now it allows us to perform the operation on library management system according to our need for example

- 1.addbook.
- 2.issue book.
- 3.Submit book.
- 4.delete book.
- 5.display book.

In Vs terminal:

```

PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    SQL CONSOLE
Python: Main + ~ [ ] [ ] ... x x
PS C:\Users\darafa> & C:\Users\darafa\AppData/Local/Programs/Python/Python312/python.exe c:/Users/darafa/Desktop/p/Main.py
Connected to MySQL database!
Enter Username:root
Your password is: 71270
Enter password71270
.....LIBRARY MANAGEMENT.....
1. Add book
2. Issues book
3. Submitt book
4. Delete book
5. Display book

Enter task no:1
-----
Enter book name: java
Enter book code:101
Total books: 12
Enter subject:computers
-----
Data entered successfully

```

In command prompt:

```
mysql> select *from book;
+-----+-----+-----+-----+
| bname | bcode | total | subject |
+-----+-----+-----+-----+
| asd    | fvk   | 0     | fkl     |
| java   | 101   | 12    | computers |
| science | 102   | 13    | medical  |
| urdu   | 104   | 20    | arts     |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

FUTURE ADVANCEMENT

In library management system, it could include:

1. Artificial intelligence integration.
2. Block chain for security.
3. Internet of things sensors.
4. Enhanced accessibility features .
5. Mobile app development.
6. Data analytics for decision making.
7. Virtual and augmented reality.
8. Open access initiative.
9. Enhanced inter library loan services.
10. Personalized user experiences.

CONCLUSION

This Library Management System embodies a fundamental yet efficient fusion of Python, MySQL, and command-line interactions. Its systematic functions and database operations provide a user-friendly platform for managing book inventory, recording transactions, and ensuring precision in library operations. This system serves as a robust foundation for future enhancements and scalability in more sophisticated library management solutions.